

# Completion Is Not Correctness: Verifying LLM Agents for Molecular Visualization

Peter [Surname]\* Diego [Surname]

Theoretical and Computational Biophysics Group / Beckman Institute

pinhao.peter@gmail.com

June 18, 2026

## Abstract

Large language model (LLM) agents can now drive scientific visualization tools such as VMD by emitting Tcl commands, and recent benchmarks score them by *task completion*: did the agent produce an answer or render a scene? We argue that for scientific use this is the wrong target. We present **vmdbench**, a correctness-oriented benchmark that checks each agent answer against an independently computed *oracle* value (cross-validated against MDAnalysis and Biopython), and we use it to measure the gap between completion and correctness for an open-weight agent (Qwen2.5-72B). The agent almost always *completes* tasks while being correct far less often — it confidently reports plausible-but-wrong numbers and builds empty visualizations that pass a hollow rubric. We then show that three targeted interventions — proactive reference injection, Tcl pre-validation, and semantic measurement tools — raise correctness to 97% and collapse the completion–correctness gap to roughly zero on the analysis suite. Finally, through a held-out evaluation on operations the tools do *not* cover, we decompose that gain honestly: most of it is tool coverage rather than a general capability jump, with a smaller, idiom-specific improvement attributable to injection. We release the benchmark, oracles, and agent harness.

## 1 Introduction

LLM agents are increasingly proposed as natural-language front-ends to scientific software: a researcher asks for a radius of gyration or a licorice representation, and the agent writes the Tcl or Python that the tool executes. Evaluations of such agents, including SciVisAgentBench-style suites, predominantly reward *completion* — the agent ran to termination and produced *an* answer or *a* rendered image. For a chatbot this is a reasonable proxy. For a scientific instrument it is dangerous: a confidently reported wrong number is worse than a refusal, and a visualization that “looks done” but contains zero representations is a silent failure.

This paper makes a simple, measurable point: **completion is not correctness**, and the distance between them is large enough to change conclusions. We build a benchmark that measures correctness directly, quantify the gap for a strong open-weight agent, and then close most of it — while being explicit about *why* it closes, separating genuine generalization from the mechanical effect of giving the agent tools that happen to match the benchmark’s metrics.

Our contributions are: (i) **vmdbench**, a correctness benchmark for VMD agents built on independently computed, cross-validated oracle values for both analysis and visualization tasks; (ii) a measurement of the *completion–correctness gap* for Qwen2.5-72B across five protein structures; (iii) three interventions that raise correctness to 97%; and (iv) a held-out rigor study that decomposes the improvement into tool coverage versus generalization.

---

\*Corresponding author.

## 2 vmdbench: measuring correctness, not completion

Each task pairs a natural-language prompt with a deterministic *oracle*: a script that computes the gold value directly from the structure, independent of the agent. For analysis tasks (radius of gyration, SASA, atom and residue counts,  $C\alpha$  distances, RMSD) the oracle emits a scalar that we compare to the agent’s answer within a metric-specific tolerance. To guard against oracle bugs we cross-validate the gold against two independent libraries, MDAnalysis and Biopython; 19 of 20 metrics agree to tolerance, and the lone discrepancy is a known definitional difference rather than an error.

Crucially, we also verify *visualization* tasks, where completion-style scoring is weakest. Instead of asking “did an image appear,” we assert on the resulting scene state — whether the requested representation actually exists, whether the named style and coloring were applied, and whether a non-empty image was written. This catches the failure mode where an agent claims to have built a “licorice representation” but created zero representations.

## 3 The completion–correctness gap

We evaluate Qwen2.5-72B (AWQ 4-bit, served locally) as a VMD agent across five structures ranging from 327 to over 9,000 atoms. The agent is a near-perfect *completer*: it terminates and reports a value on almost every task. Its *correctness*, however, is far lower. The agent recomputes self-RMSD as a nonzero value when it should be exactly zero, reports plausible but out-of-tolerance radii of gyration, and — on visualization tasks — produces scenes that satisfy a hollow rubric while containing no representations. The gap between completion and correctness is not noise; it is the central measurement, and a benchmark that scores only completion would report this agent as strong.

## 4 Closing the gap

Three interventions, each motivated by an observed failure mode, move the agent from “completes everything, correct sometimes” to “correct when it answers.”

**(1) Proactive reference injection.** The agent rarely retrieves documentation even when instructed to. Rather than rely on agent-initiated retrieval, we inject a small, curated VMD-Tcl cheat-sheet into the context up front (a DocPrompting-style move). This fixes idiom-level errors such as the self-RMSD recomputation.

**(2) Tcl pre-validation.** The agent occasionally emits syntactically incomplete Tcl (unbalanced braces or brackets), which can hang the interpreter. We reject such commands before execution with a lightweight balance check and raise per-command timeouts.

**(3) Semantic measurement tools.** We expose typed tools (`vmd_measure`, `vmd_represent`) that emit the correct Tcl for a named operation, so the model only has to choose the operation, not author the script.

Together these raise correctness to **97%** on the analysis suite, with the completion–correctness gap collapsing to approximately zero: when the improved agent answers, it is right.

## 5 What actually generalizes

A 97% number invites an obvious objection: the semantic tools’ operations *are* the benchmark’s metrics, so some of the gain is the tool doing the task by construction. We test this directly on a **held-out** set of six operations the tools do *not* cover — hydrogen bonds, a three-atom angle, a backbone dihedral, center of mass,

bounding-box extent, and a filtered glycine count — requiring the agent to write raw Tcl. We compare three arms: the raw agent (NONE), reference injection only (INJECT), and the full system (+TOOLS); each cell is a pass rate over two structures and three seeds (Table 1).

| held-out operation  | NONE         | INJECT       | +TOOLS       |
|---------------------|--------------|--------------|--------------|
| hydrogen bonds      | 6/6          | 6/6          | 6/6          |
| 3-atom angle        | 4/6          | 6/6          | 6/6          |
| dihedral ( $\phi$ ) | 3/6          | 6/6          | 6/6          |
| center of mass      | 6/6          | 6/6          | 6/6          |
| bounding-box extent | 2/6          | 0/6          | 0/6          |
| glycine count       | 0/6          | 0/6          | 0/6          |
| <b>total</b>        | <b>21/36</b> | <b>24/36</b> | <b>24/36</b> |

Table 1: Held-out correctness on tool-uncovered operations. INJECT equals +TOOLS, so the semantic tools add nothing off their coverage; the only generalization is from injection, and only on the idioms the cheat-sheet teaches.

Three things follow. First, the headline 97% is *largely tool coverage*: stripped of the covering tools, correctness falls to 67% (24/36). Second, the semantic tools add *nothing* off their coverage — INJECT and +TOOLS are identical — so they are an engineering win when they cover the operation and neutral otherwise. Third, the real generalization comes from *injection* (21→24), and only on the idioms the cheat-sheet teaches: the three-atom angle (4→6) and the dihedral (3→6). Operations the cheat-sheet never mentions are either solved regardless (hydrogen bonds, center of mass) or resist every arm (glycine count). Injection is not free: it slightly *hurt* the bounding-box task (2→0), evidence that an injected reference can distract on operations it does not cover.

## 6 Limitations

The held-out study uses two structures and three seeds per cell; the small denominators mean single-task movements (e.g., bounding-box extent) should be read as suggestive, not definitive. Our oracles encode specific operational definitions (e.g., hydrogen-bond geometry, RMSD against self), and an agent using a different-but-valid convention may be scored wrong; we mitigate this with cross-validation but do not eliminate it. Finally, results are reported for a single open-weight model; multi-model baselines are in progress and would strengthen the claim that the completion–correctness gap is a property of the task formulation rather than of one model.

## 7 Conclusion

For scientific agents, scoring completion rewards confident wrongness. We show the gap is large and measurable, that targeted interventions can close it on covered operations, and — being honest about it — that most of the closing is tool coverage rather than a general capability jump. We advocate correctness-oriented, oracle-backed evaluation as the default for agents that drive scientific instruments, and release vmdbench toward that end.

**Reproducibility.** The benchmark, oracle scripts, cross-validation harness, agent adapter, and held-out runner are released with deterministic seeds and per-task gold values.